Integrating Wazuh with AbuseIPDB

Goal of This Integration

In this implementation, my goal was to enhance the detection capability of Wazuh by integrating it with AbuseIPDB, which provides real-time IP reputation intelligence.

The main objective of this integration is to detect suspicious SSH login attempts and automatically verify whether the source IP address is already reported as malicious in a global threat database. If the IP address is confirmed as abusive, Wazuh generates a higher-severity alert so that immediate action can be taken.

This integration helps move from simple log monitoring to intelligence-based threat detection.

What is AbuseIPDB?

AbuseIPDB is an open and collaborative threat intelligence platform that collects reports about malicious IP addresses from security professionals and organizations worldwide.

It maintains a database of IP addresses involved in activities such as:

- SSH brute-force attacks
- Spam and email abuse
- Port scanning
- Exploitation attempts
- Malware communication

For each IP address, AbuseIPDB provides important details such as:

- Abuse Confidence Score (0_100)
- Total number of reports
- Country and ISP information
- Usage type (hosting, residential, etc.)
- Last reported activity

This information helps determine whether an IP address is safe or potentially dangerous.

Why I Integrated AbuseIPDB with Wazuh

Wazuh is capable of detecting suspicious activities such as failed SSH login attempts. However, by default, it does not know whether the attacking IP address has a previous malicious history.

To solve this limitation, I integrated AbuseIPDB with Wazuh so that every suspicious external IP can be verified automatically.

With this integration:

- Wazuh detects suspicious SSH activity
- The source IP is sent to AbuseIPDB
- AbuseIPDB returns the reputation score
- Wazuh enriches the alert with this intelligence

This allows better decision-making and reduces unnecessary alerts.

What I Achieved Through This Integration

After completing the integration, I achieved the following improvements:

- Automatic verification of attacker IP addresses
- Reduction of false positive alerts
- Prioritization of high-risk IP addresses
- Enriched alerts with global intelligence data
- Improved response time for security incidents

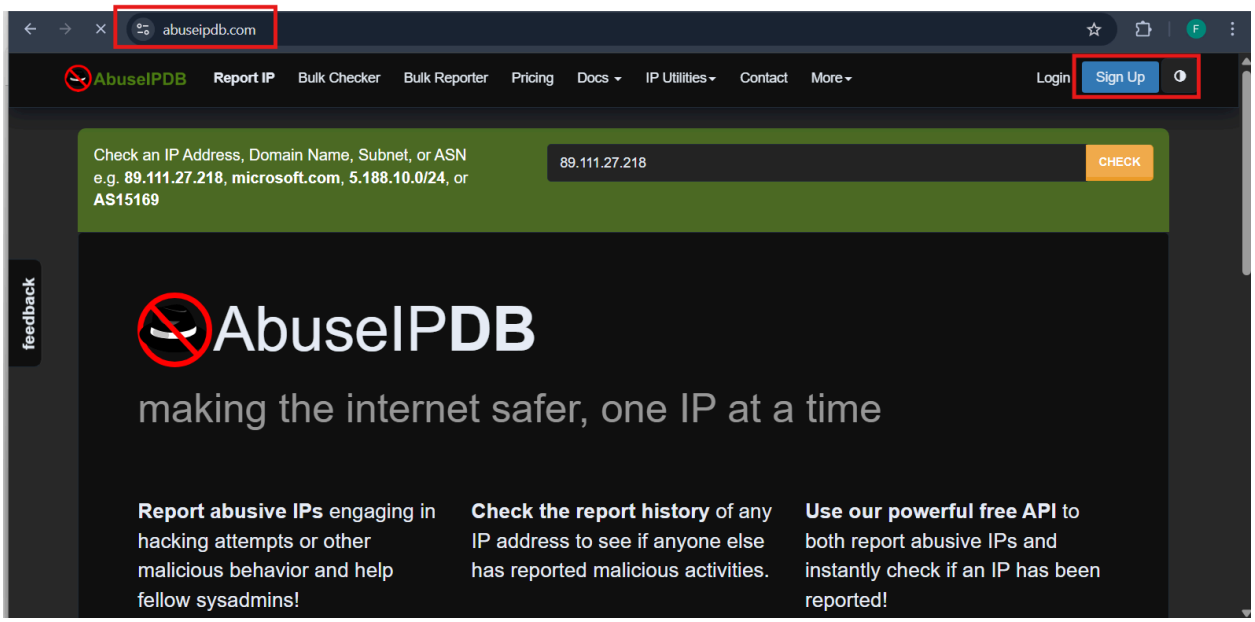
Overall, this integration made the SIEM system more intelligent and practical for real-world attack detection.

Step-by-Step Implementation

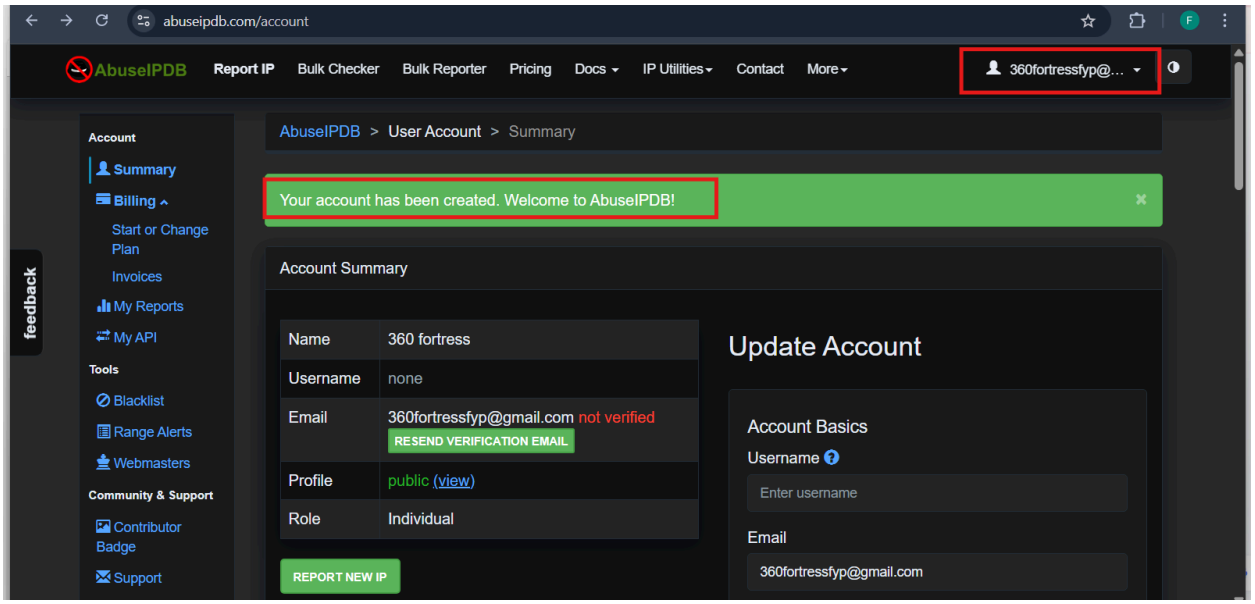
Step 1: Creating an AbuseIPDB Account

First, I created an account on the AbuseIPDB website.

1. I visited: https://www.abuseipdb.com
2. Clicked on **Sign Up**



3. Registered using my email address
4. Verified my account
5. Logged in successfully

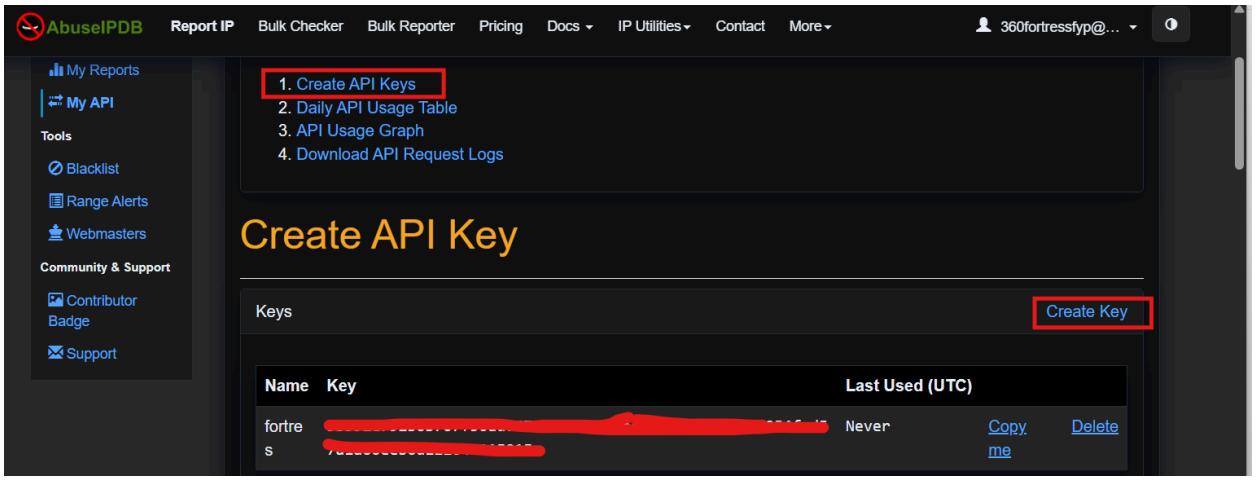


Step 2: Generating the API Key

After logging in:

1. I went to the profile section
2. Opened the **API** tab
3. Generated a new API key

4. Copied and saved the key securely



This API key is required because Wazuh uses it to authenticate requests to AbuseIPDB.

Step 3: Creating the Custom Integration Script

On the Wazuh Manager (Ubuntu), I navigated to the integrations directory:

cd /var/ossec/integrations

```
wazuh@fypserver:~$ cd /var/ossec/integrations
wazuh@fypserver:/var/ossec/integrations$
```

Then I created a custom script:

sudo nano custom-abuseipdb.py

```
wazuh@fypserver:/var/ossec/integrations$ sudo nano custom-abuseipdb.py
wazuh@fypserver:/var/ossec/integrations$
```

#!/var/ossec/framework/python/bin/python3

import json

import sys

import time

import os

from socket import socket, AF_UNIX, SOCK_DGRAM

```
try:
    import requests
    from requests.auth import HTTPBasicAuth
except Exception as e:
    print("No module 'requests' found. Install: pip install requests")
    sys.exit(1)

# Global vars

debug_enabled = False
pwd = os.path.dirname(os.path.dirname(os.path.realpath(__file__)))
json_alert = {}
now = time.strftime("%a %b %d %H:%M:%S %Z %Y")

# Set paths
log_file = '{0}/logs/integrations.log'.format(pwd)
socket_addr = '{0}/queue/sockets/queue'.format(pwd)

def main(args):
    debug("# Starting")

    # Read args
    alert_file_location = args[1]
    apikey = args[2]
```

```
    debug("# API Key")
    debug(apikey)

    debug("# File location")
    debug(alert_file_location)

    # Load alert. Parse JSON object.
    with open(alert_file_location) as alert_file:
        json_alert = json.load(alert_file)
    debug("# Processing alert")
    debug(json_alert)

    # Request AbuseIPDB info
    msg = request_abuseipdb_info(json_alert,apikey)

    # If positive match, send event to Wazuh Manager
    if msg:
        send_event(msg, json_alert["agent"])

def debug(msg):
    if debug_enabled:
        msg = "{0}: {1}\n".format(now, msg)

        print(msg)
```

```
        f = open(log_file,"a")
        f.write(msg)
        f.close()


def collect(data):
  abuse_confidence_score = data['abuseConfidenceScore']
  country_code = data['countryCode']
  usage_type = data['usageType']
  isp = data['isp']
  domain = data['domain']
  total_reports = data['totalReports']
  last_reported_at = data['lastReportedAt']
  return abuse_confidence_score, country_code, usage_type, isp, domain, total_reports,
last_reported_at


def in_database(data, srcip):
  result = data['totalReports']
  if result == 0:
    return False
  return True


def query_api(srcip, apikey):
  params = {'maxAgeInDays': '90', 'ipAddress': srcip,}
  headers = {
  "Accept-Encoding": "gzip, deflate",
```

```
  'Accept': 'application/json',

  "Key": apikey

  }

  response = requests.get('https://api.abuseipdb.com/api/v2/check',params=params,
headers=headers)

  if response.status_code == 200:

      json_response = response.json()

      data = json_response["data"]

      return data

  else:

      alert_output = {}

      alert_output["abuseipdb"] = {}

      alert_output["integration"] = "custom-abuseipdb"

      json_response = response.json()

      debug("# Error: The AbuseIPDB encountered an error")

      alert_output["abuseipdb"]["error"] = response.status_code

      alert_output["abuseipdb"]["description"] = json_response["errors"][0]["detail"]

      send_event(alert_output)

      exit(0)


def request_abuseipdb_info(alert, apikey):

    alert_output = {}

    # If there is no source ip address present in the alert. Exit.

    if not "srcip" in alert["data"]:


        return(0)
```

```
    # Request info using AbuseIPDB API
    data = query_api(alert["data"]["srcip"], apikey)

    # Create alert
    alert_output["abuseipdb"] = {}
    alert_output["integration"] = "custom-abuseipdb"
    alert_output["abuseipdb"]["found"] = 0
    alert_output["abuseipdb"]["source"] = {}
    alert_output["abuseipdb"]["source"]["alert_id"] = alert["id"]
    alert_output["abuseipdb"]["source"]["rule"] = alert["rule"]["id"]
    alert_output["abuseipdb"]["source"]["description"] = alert["rule"]["description"]
    alert_output["abuseipdb"]["source"]["full_log"] = alert["full_log"]
    alert_output["abuseipdb"]["source"]["srcip"] = alert["data"]["srcip"]
    srcip = alert["data"]["srcip"]
    # Check if AbuseIPDB has any info about the srcip
    if in_database(data, srcip):
      alert_output["abuseipdb"]["found"] = 1

    # Info about the IP found in AbuseIPDB
    if alert_output["abuseipdb"]["found"] == 1:
        abuse_confidence_score, country_code, usage_type, isp, domain, total_reports,
last_reported_at = collect(data)

        # Populate JSON Output object with AbuseIPDB request
        alert_output["abuseipdb"]["abuse_confidence_score"] = abuse_confidence_score
```

```
        alert_output["abuseipdb"]["country_code"] = country_code
        alert_output["abuseipdb"]["usage_type"] = usage_type
        alert_output["abuseipdb"]["isp"] = isp
        alert_output["abuseipdb"]["domain"] = domain
        alert_output["abuseipdb"]["total_reports"] = total_reports
        alert_output["abuseipdb"]["last_reported_at"] = last_reported_at

    debug(alert_output)

    return(alert_output)

def send_event(msg, agent = None):
    if not agent or agent["id"] == "000":
        string = '1:abuseipdb:{0}'.format(json.dumps(msg))
    else:
        string = '1:[{0}] ({1}) {2}->abuseipdb:{3}'.format(agent["id"], agent["name"],
agent["ip"] if "ip" in agent else "any", json.dumps(msg))

    debug(string)
    sock = socket(AF_UNIX, SOCK_DGRAM)
    sock.connect(socket_addr)
    sock.send(string.encode())
    sock.close()

if __name__ == "__main__":
    try:
```

```
        # Read arguments
        bad_arguments = False
        if len(sys.argv) >= 4:
            msg = '{0} {1} {2} {3} {4}'.format(now, sys.argv[1], sys.argv[2], sys.argv[3],
sys.argv[4] if len(sys.argv) > 4 else '')
            debug_enabled = (len(sys.argv) > 4 and sys.argv[4] == 'debug')
        else:
            msg = '{0} Wrong arguments'.format(now)
            bad_arguments = True

        # Logging the call
        f = open(log_file, 'a')
        f.write(msg +'\n')
        f.close()

        if bad_arguments:
            debug("# Exiting: Bad arguments.")
            sys.exit(1)

        # Main function
        main(sys.argv)

    except Exception as e:
        debug(str(e))
        Raise
```

```
  GNU nano 6.2                                custom-abuseipdb.py *
import json
import sys
import time
import os
from socket import socket, AF_UNIX, SOCK_DGRAM

try:
    import requests
    from requests.auth import HTTPBasicAuth
except Exception as e:
    print("No module 'requests' found. Install: pip install requests")
    sys.exit(1)

# Global vars

debug_enabled = False
pwd = os.path.dirname(os.path.dirname(os.path.realpath(__file__)))
json_alert = {}
now = time.strftime("%a %b %d %H:%M:%S %Z %Y")

# Set paths
log_file = '{0}/logs/integrations.log'.format(pwd)
socket_addr = '{0}/queue/sockets/queue'.format(pwd)

def main(args):
    debug("# Starting")
```

This script performs the following tasks:

- Reads the alert generated by Wazuh
- Extracts the source IP address
- Sends the IP address to AbuseIPDB using the API
- Receives the abuse confidence score
- Sends enriched data back to Wazuh

This script acts as a bridge between Wazuh and AbuseIPDB.

Step 4: Setting Proper Permissions

After creating the script, I assigned the correct permissions:

sudo chmod 750 /var/ossec/integrations/custom-abuseipdb.py

sudo chown root:wazuh /var/ossec/integrations/custom-abuseipdb.py

```
wazuh@fypserver:/var/ossec/integrations$ sudo chmod 750 /var/ossec/integrations/custom-abuseipdb.py
sudo chown root:wazuh /var/ossec/integrations/custom-abuseipdb.py
wazuh@fypserver:/var/ossec/integrations$
```

This ensures that the script can be executed securely by Wazuh.

Step 5: Configuring Integration in Wazuh

Next, I modified the Wazuh configuration file:

sudo nano /var/ossec/etc/ossec.conf

```
wazuh@fypserver:~$ sudo nano /var/ossec/etc/ossec.conf
[sudo] password for wazuh:
wazuh@fypserver:~$
```

Inside the <ossec_config> section, I added the following block:

<integration>

<name>custom-abuseipdb.py</name>

<hook_url>https://api.abuseipdb.com/api/v2/check</hook_url>

<api_key>MY_ABUSEIPDB_API_KEY</api_key>

<rule_id>100002,100003</rule_id>

<alert_format>json</alert_format>

</integration>

```
GNU nano 6.2                                    /var/ossec/etc/ossec.conf *
<command>
  <name>win_route-null</name>
  <executable>route-null.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>netsh</name>
  <executable>netsh.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
  <!-- Abusipdb Integration -->
<integration>
 <name>custom-abuseipdb.py</name>
 <hook_url>https://api.abuseipdb.com/api/v2/check</hook_url>
 <api_key>[redacted]</api_key>
 <rule_id>100002,100003</rule_id>
 <alert_format>json</alert_format>
</integration>
```

Then I replaced MY_ABUSEIPDB_API_KEY with my actual API key.

This configuration tells Wazuh to execute the custom script whenever the specified SSH rules are triggered.

Step 6: Defining Custom SSH Rules

I edited the local rules file:

sudo nano /var/ossec/etc/rules/local_rules.xml

```
wazuh@fypserver:~$ sudo nano /var/ossec/etc/rules/local_rules.xml
wazuh@fypserver:~$
```

Here, I created custom rules to detect:

- SSH authentication failed from a public IP
- SSH authentication succeeded from a public IP

I used regex filtering to ignore private IP ranges (such as 192.168.x.x or 10.x.x.x) so that only external IP addresses are checked with AbuseIPDB.

```
<group name="local,syslog,sshd,">

 <rule id="100002" level="5">

    <if_sid>5716</if_sid>

    <match
type="pcre2">\b(?!(10)|192\.168|172\.(2[0-9]|1[6-9]|3[0-1])|(25[6-9]|2[6-9][0-9]|[3-9][0-9][
0-9]|99[1-9]))[0-9]{1,3}\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-
9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)</match>

    <description>sshd: Authentication failed from a public IP address
$(srcip).</description>


<group>authentication_failed,authentication_success,pci_dss_10.2.4,pci_dss_10.2.5,</
group>

  </rule>



  <rule id="100003" level="5">

    <if_sid>5715</if_sid>

    <match
type="pcre2">\b(?!(10)|192\.168|172\.(2[0-9]|1[6-9]|3[0-1])|(25[6-9]|2[6-9][0-9]|[3-9][0-9][
0-9]|99[1-9]))[0-9]{1,3}\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-
9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)</match>

    <description>sshd: Authentication succeeded from a public IP address
$(srcip).</description>
```

```
<group>authentication_failed,authentication_success,pci_dss_10.2.4,pci_dss_10.2.5,</
group>

  </rule>

</group>
```

```
GNU nano 6.2                    /var/ossec/etc/rules/local_rules.xml *

<group name="local,syslog,sshd,">
  <rule id="100002" level="5">
    <if_sid>5716</if_sid>
    <match type="pcre2">\b(?!(10)|192\.168|172\.(2[0-9]|1[6-9]|3[0-1])|(25[6-9]|2[6-9][0-9]|[3-9][0-9][0-9]|99[1-9]))[0-9]{1,>
    <description>sshd: Authentication failed from a public IP address $(srcip).</description>
    <group>authentication_failed,authentication_success,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

  <rule id="100003" level="5">
    <if_sid>5715</if_sid>
    <match type="pcre2">\b(?!(10)|192\.168|172\.(2[0-9]|1[6-9]|3[0-1])|(25[6-9]|2[6-9][0-9]|[3-9][0-9][0-9]|99[1-9]))[0-9]{1,>
    <description>sshd: Authentication succeeded from a public IP address $(srcip).</description>
    <group>authentication_failed,authentication_success,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>
```

Step 7: Adding AbuseIPDB Result Handling Rules

After that, I added additional rules to process the AbuseIPDB results.

If the abuse confidence score is greater than zero, Wazuh generates a higher-severity alert. This ensures that only IPs with confirmed malicious history receive priority attention.

```
<group name="local,syslog,sshd,">
  <rule id="100004" level="10">
    <field name="abuseipdb.source.rule" type="pcre2">^100002$</field>
    <field name="abuseipdb.abuse_confidence_score" type="pcre2"
negate="yes">^0$</field>
    <description>AbuseIPDB: SSH Authentication failed from a public IP address
$(abuseipdb.source.srcip) with $(abuseipdb.abuse_confidence_score)% confidence of
abuse.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
  <rule id="100005" level="14">
    <field name="abuseipdb.source.rule" type="pcre2">^100003$</field>
    <field name="abuseipdb.abuse_confidence_score" type="pcre2"
negate="yes">^0$</field>
    <description>AbuseIPDB: SSH Authentication succeeded from a public IP address
$(abuseipdb.source.srcip) with $(abuseipdb.abuse_confidence_score)% confidence of
abuse.</description>
```

```
  <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
 </rule>
</group>
```

```
GNU nano 6.2                    /var/ossec/etc/rules/local_rules.xml *

<group name="local,syslog,sshd,">
  <rule id="100004" level="10">
    <field name="abuseipdb.source.rule" type="pcre2">^100002$</field>
    <field name="abuseipdb.abuse_confidence_score" type="pcre2" negate="yes">^0$</field>
    <description>AbuseIPDB: SSH Authentication failed from a public IP address $(abuseipdb.source.srcip) with $(abuseipdb.abu>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
  <rule id="100005" level="14">
    <field name="abuseipdb.source.rule" type="pcre2">^100003$</field>
    <field name="abuseipdb.abuse_confidence_score" type="pcre2" negate="yes">^0$</field>
    <description>AbuseIPDB: SSH Authentication succeeded from a public IP address $(abuseipdb.source.srcip) with $(abuseipdb.>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>
```

Step 8: Restarting Wazuh Manager

After completing all configurations, I restarted the Wazuh manager:

```
sudo systemctl restart wazuh-manager
```

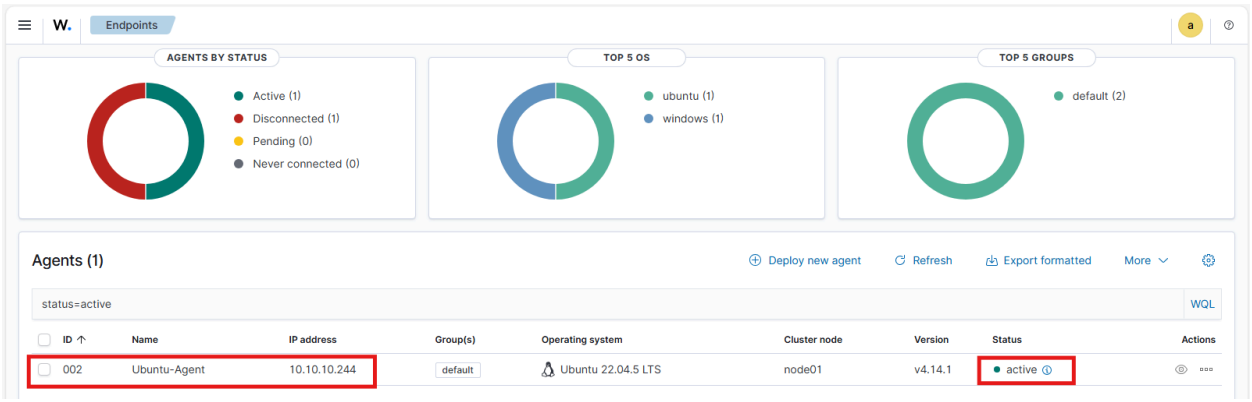
```
wazuh@fypserver:~$ sudo systemctl restart wazuh-manager
wazuh@fypserver:~$
```

This activates the integration.

Configure Agent (Ubuntu) to Feed Logs

First I checked that my agent and manager are on the same network.
So I checked my agent ip as by seeing this i am sure that my agent and manager are on the same network.
So agents and managers are connected.

```
ubuntu@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    link/ether 08:00:27:ea:49:f9 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.244/24 brd 10.10.10.255 scope global dynamic noprefixroute enp
0s3
       valid_lft 86296sec preferred_lft 86296sec
    inet6 fe80::938e:c0cc:c16c:736d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    link/ether 08:00:27:8b:b8:fb brd ff:ff:ff:ff:ff:ff
    inet6 fe80::8806:5d1b:9cdb:d87a/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
ubuntu@ubuntu:~$
```



a. Switch to Root:

sudo -i

b. Create a Log File for Testing:

touch /var/log/abuseipdb.log

```
ubuntu@ubuntu:~$  sudo -i
[sudo] password for ubuntu:
root@ubuntu:~#  touch /var/log/abuseipdb.log
root@ubuntu:~#
```

c. Stop Wazuh Agent Temporarily:

systemctl stop wazuh-agent

```
root@ubuntu:~#  systemctl stop wazuh-agent
root@ubuntu:~#
```

d. Edit Agent Config:

nano /var/ossec/etc/ossec.conf

```
root@ubuntu:~#  nano /var/ossec/etc/ossec.conf
root@ubuntu:~#
```

Add this before </ossec_config>:

<localfile>

<log_format>syslog</log_format>

<location>/var/log/abuseipdb.log</location>

</localfile>

```
GNU nano 6.2                                    /var/ossec/etc/ossec.conf *

<!-- Abuseipdb Logs File -->
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/abuseipdb.log</location>
</localfile>
```

e. Restart the Agent:

systemctl start wazuh-agent

```
root@ubuntu:~#  systemctl start wazuh-agent
root@ubuntu:~#
```

Inject Sample Log Events for Testing

Create a file with test SSH events:

nano injector

```
root@ubuntu:~# nano injector
root@ubuntu:~#
```

Dec 10 01:02:02 host sshd[1234]: Accepted password for root from 64.62.197.132 port 1066 ssh2

Dec 10 01:02:02 host sshd[1234]: Failed none for root from 116.25.251.248 port 1066 ssh2

```
  GNU nano 6.2                                          injector
Dec 10 01:02:02 host sshd[1234]: Accepted password for root from 64.62.197.132 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Failed none for root from 116.25.251.248 port 1066 ssh2
```

Append to the monitored log:

cat injector >> /var/log/abuseipdb.log

```
root@ubuntu:~# cat injector >> /var/log/abuseipdb.log
root@ubuntu:~#
```

Check the file:

tail /var/log/abuseipdb.log

```
root@ubuntu:~# tail /var/log/abuseipdb.log
Dec 10 01:02:02 host sshd[1234]: Failed password for root from 116.25.251.248 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Accepted password for root from 64.62.197.132 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Failed password for root from 116.25.251.248 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Accepted password for root from 64.62.197.132 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Accepted password for root from 64.62.197.132 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Failed none for root from 116.25.251.248 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Accepted password for root from 64.62.197.132 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Failed none for root from 116.25.251.248 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Accepted password for root from 64.62.197.132 port 1066 ssh2
Dec 10 01:02:02 host sshd[1234]: Failed none for root from 116.25.251.248 port 1066 ssh2
root@ubuntu:~#
```

We also manually check the confidence score:

curl -G https://api.abuseipdb.com/api/v2/check --data-urlencode "ipAddress=116.25.251.248" -d maxAgeInDays=90 -H "Key: My_API_KEY" -H "Accept: application/json"

```
wazuh@fypserver:~$ curl -G https://api.abuseipdb.com/api/v2/check   --data-urlencode "ipAddress=87.236.176.50"   -d maxAgeInDa
ys=90   -H "Key: 9b392e792b6b707f50ad6675b499cb0deeee02569126bc240054fed57a1a60eeb6d121044115915a"   -H "Accept: application/j
son"
{"data":{"ipAddress":"87.236.176.50","isPublic":true,"ipVersion":4,"isWhitelisted":false,"abuseConfidenceScore":100,"countryCo
de":"GB","usageType":"Fixed Line ISP","isp":"Driftnet Ltd","domain":"driftnet.io","hostnames":["joyful.monitoring.internet-mea
wazuh@fypserver:~$ ourl -G https://api.abuseipdb.com/api/v2/check   --data-urlencode "ipAddress=85.62.67.73"   -d maxAgeInDays
=90   -H "Key: 9b392e792b6b707f50ad6675b499cb0deeee02569126bc240054fed57a1a60eeb6d121044115915a"   -H "Accept: application/jso
n"
{"data":{"ipAddress":"85.62.67.73","isPublic":true,"ipVersion":4,"isWhitelisted":false,"abuseConfidenceScore":0,"countryCode":
"ES","usageType":"Mobile ISP","isp":"Red CANAL MARBELLA TELEVISION SL","domain":"masorange.es","hostnames":["73.pool85-62-67.d
ynamic.orange.es"],"isTor":false,"totalReports":0,"numDistinctUsers":0,"lastReportedAt":"2025-09-09T12:38:36+00:00"}}wazuh@fyp
server:~$ curl -G https://api.abuseipdb.com/api/v2/check   --data-urlencode "ipAddress=8^CH "Accept: application/json"
wazuh@fypserver:~$ curl -G https://api.abuseipdb.com/api/v2/check   --data-urlencode "ipAddress=116.25.251.248"   -d maxAgeInD
ays=90   -H "Key: 9b392e792b6b707f50ad6675b499cb0deeee02569126bc240054fed57a1a60eeb6d121044115915a"   -H "Accept: application/
json"
{"data":{"ipAddress":"116.25.251.248","isPublic":true,"ipVersion":4,"isWhitelisted":false,"abuseConfidenceScore":57,"countryCo
de":"CN","usageType":"Fixed Line ISP","isp":"CHINANET Guangdong province network","domain":"chinatelecom.cn","hostnames":[],"i
sTor":false,"totalReports":9,"numDistinctUsers":9,"lastReportedAt":"2026-02-24T23:46:18+00:00"}}wazuh@fypserver:~$
wazuh@fypserver:~$ ^C
wazuh@fypserver:~$
```

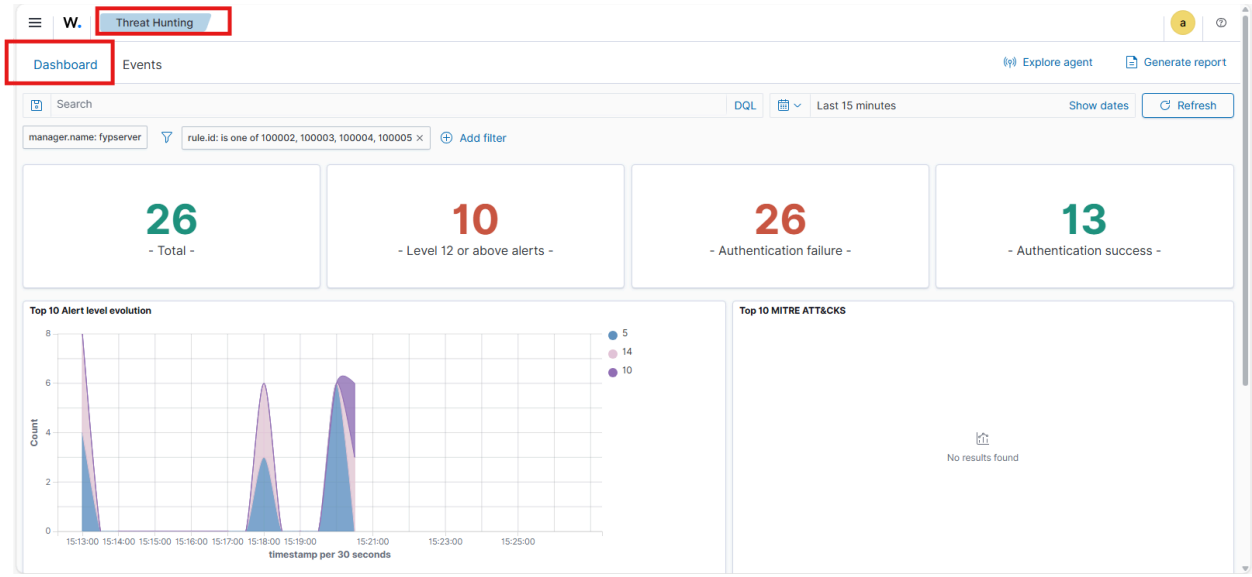
Monitor Results in Wazuh Dashboard

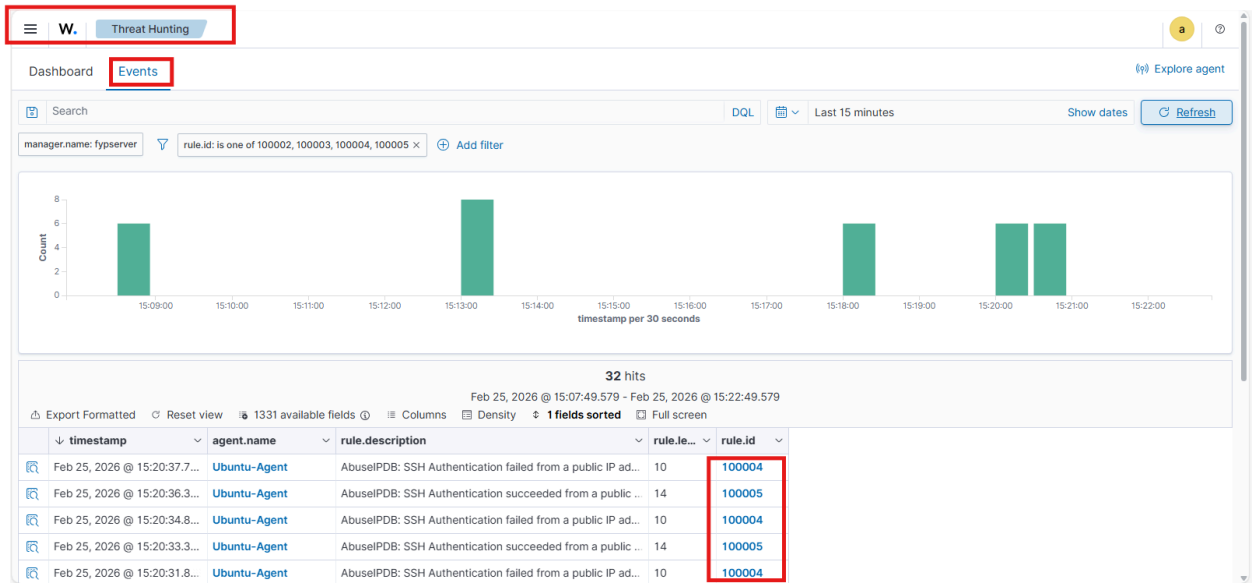
In the Wazuh dashboard, I verified the alert under the Threat Hunting section. The alert included:

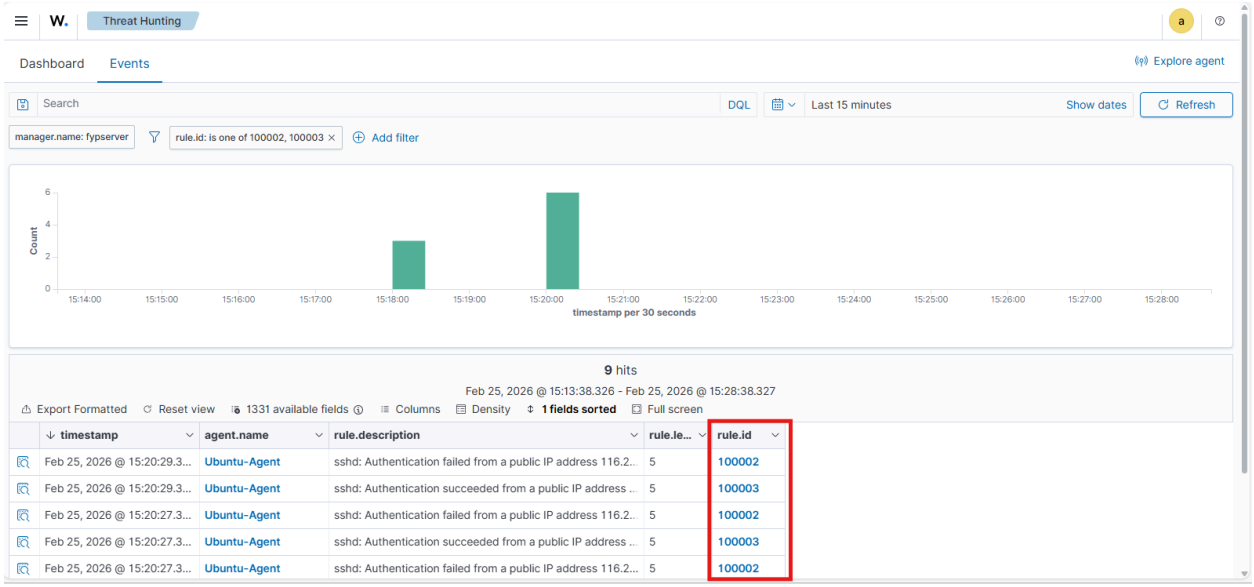
- rule.id:100004 or 100005
- rule.id: 100002 OR rule.id: 100003 OR rule.id: 100004 OR rule.id: 100005

- Source IP address
- Abuse confidence score
- Country and ISP details
- SSH log information

This confirmed that the integration was working successfully.

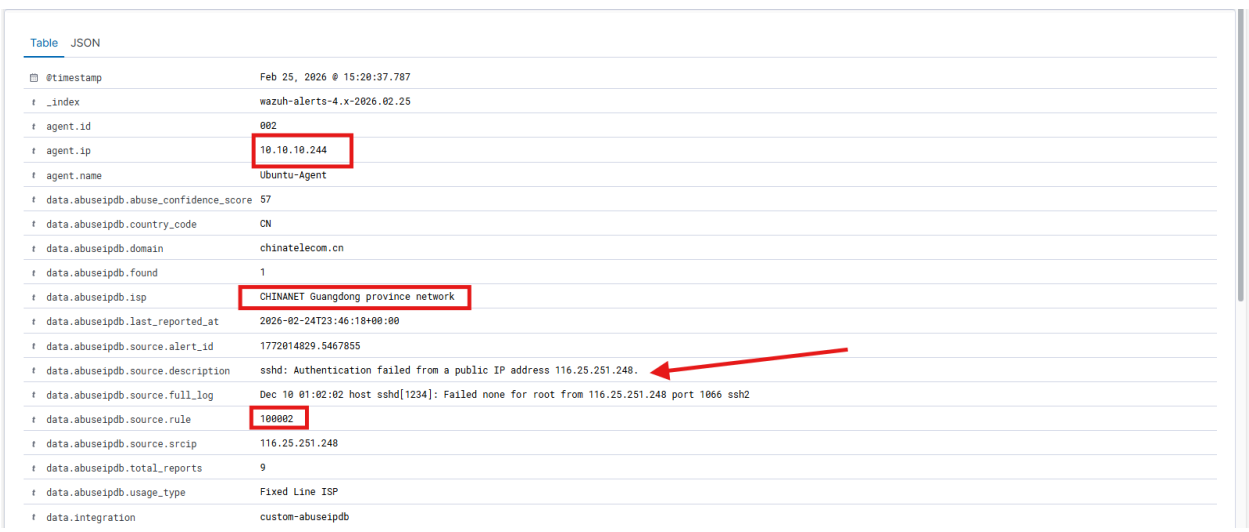

Threat Hunting
Dashboard
Events
Explore agent
Generate report
Search
DQL
Last 15 minutes
Show dates
Refresh
manager.name: fypserver
rule.id: is one of 100002, 100003, 100004, 100005
Add filter
26
- Total -
10
- Level 12 or above alerts -
26
- Authentication failure -
13
- Authentication success -
Top 10 Alert level evolution
Count
timestamp per 30 seconds
5
14
10
Top 10 MITRE ATT&CKS
No results found


Threat Hunting
Dashboard
Events
Explore agent
Search
DQL
Last 15 minutes
Show dates
Refresh
manager.name: fypserver
rule.id: is one of 100002, 100003, 100004, 100005
Add filter
Count
timestamp per 30 seconds
32 hits
Feb 25, 2026 @ 15:07:49.579 - Feb 25, 2026 @ 15:22:49.579
Export Formatted
Reset view
1331 available fields
Columns
Density
1 fields sorted
Full screen
timestamp
agent.name
rule.description
rule.le...
rule.id
Feb 25, 2026 @ 15:20:37.7... Ubuntu-Agent AbuseIPDB: SSH Authentication failed from a public IP ad... 10 100004
Feb 25, 2026 @ 15:20:36.3... Ubuntu-Agent AbuseIPDB: SSH Authentication succeeded from a public ... 14 100005
Feb 25, 2026 @ 15:20:34.8... Ubuntu-Agent AbuseIPDB: SSH Authentication failed from a public IP ad... 10 100004
Feb 25, 2026 @ 15:20:33.3... Ubuntu-Agent AbuseIPDB: SSH Authentication succeeded from a public ... 14 100005
Feb 25, 2026 @ 15:20:31.8... Ubuntu-Agent AbuseIPDB: SSH Authentication failed from a public IP ad... 10 100004



Now we see the details of alerts that are generated after threat intelligence source:

First I see the details of Authentication failed alerts and rule id is 100004:



decoder.name	json
full_log	{"abuseipdb": {"found": 1, "source": {"alert_id": "1772014829.5467855", "rule": "100002", "description": "sshd: Authentication failed from a public IP address 116.25.251.248.", "full_log": "Dec 10 01:02:02 host sshd[1234]: Failed none for root from 116.25.251.248 port 1066 ssh2", "srcip": "116.25.251.248"}, "abuse_confidence_score": 57, "country_code": "CN", "usage_type": "Fixed Line ISP", "isp": "CHINANET Guangdong province network", "domain": "chinatelecom.cn", "total_reports": 9, "last_reported_at": "2026-02-24T23:46:18+00:00"}, "integration": "custom-abuseipdb"}
id	1772014837.5476053
input.type	log
location	abuseipdb
manager.name	fypserver
rule.description	AbuseIPDB: SSH Authentication failed from a public IP address 116.25.251.248 with 57% confidence of abuse.
rule.firedtimes	3
rule.groups	local, syslog, sshd, authentication_failed
rule.id	100004
rule.level	10
rule.mail	false
rule.pci_dss	10.2.4, 10.2.5
timestamp	Feb 25, 2026 @ 15:20:37.787

Now we see the details of Authentication success alerts and rule id is 100005:

Discover | wazuh-alerts-4.x-2026.02.25#_rZQIJwB6GHPhG1spmyB

Table JSON

@timestamp	Feb 25, 2026 @ 15:20:36.300
_index	wazuh-alerts-4.x-2026.02.25
agent.id	002
agent.ip	10.10.10.244
agent.name	Ubuntu-Agent
data.abuseipdb.abuse_confidence_score	100
data.abuseipdb.country_code	US
data.abuseipdb.domain	shadowserver.org
data.abuseipdb.found	1
data.abuseipdb.isp	The Shadowserver Foundation, Inc.
data.abuseipdb.last_reported_at	2026-02-25T09:56:00+00:00
data.abuseipdb.source.alert_id	1772014829.5467424
data.abuseipdb.source.description	sshd: Authentication succeeded from a public IP address 64.62.197.132.
data.abuseipdb.source.full_log	Dec 10 01:02:02 host sshd[1234]: Accepted password for root from 64.62.197.132 port 1066 ssh2
data.abuseipdb.source.rule	100003
data.abuseipdb.source.srcip	64.62.197.132
data.abuseipdb.total_reports	2189
data.abuseipdb.usage_type	Data Center/Web Hosting/Transit
data.integration	custom-abuseipdb

decoder.name	json
full_log	{"abuseipdb": {"found": 1, "source": {"alert_id": "1772014829.5467424", "rule": "100003", "description": "sshd: Authentication succeeded from a public IP address 64.62.197.132.", "full_log": "Dec 10 01:02:02 host sshd[1234]: Accepted password for root from 64.62.197.132 port 1066 ssh2", "srcip": "64.62.197.132"}, "abuse_confidence_score": 100, "country_code": "US", "usage_type": "Data Center/Web Hosting/Transit", "isp": "The Shadowserver Foundation, Inc.", "domain": "shadowserver.org", "total_reports": 2189, "last_reported_at": "2026-02-25T09:56:00+00:00"}, "integration": "custom-abuseipdb"}
id	1772014836.5474474
input.type	log
location	abuseipdb
manager.name	fypserver
rule.description	AbuseIPDB: SSH Authentication succeeded from a public IP address 64.62.197.132 with 100% confidence of abuse.
rule.firedtimes	17
rule.groups	local, syslog, sshd, authentication_failed
rule.id	100005
rule.level	14
rule.mail	true
rule.pci_dss	10.2.4, 10.2.5
timestamp	Feb 25, 2026 @ 15:20:36.300

Summary:

AbuseIPDB is an open threat intelligence platform that maintains a global database of IP addresses reported for malicious activities such as SSH brute-force attacks, port scanning, spam, and exploitation attempts. It provides an Abuse Confidence Score for each IP, along with country, ISP, and usage type information. Integrating AbuseIPDB with Wazuh enhances the threat detection capability of the SIEM system. When Wazuh detects a suspicious SSH login attempt from an external IP address, it automatically sends that IP to AbuseIPDB. AbuseIPDB returns the abuse confidence score, and Wazuh generates an enriched, higher-severity alert based on the result.

Through this integration, several improvements were achieved. Wazuh can now verify whether an attacking IP has a confirmed malicious history using global intelligence. Alerts are automatically enriched with reputation data, enabling faster and more accurate incident response. False positives are reduced because private and clean IPs are filtered out before checking. High-risk IPs are prioritized with elevated alert severity levels, allowing security teams to focus on real threats. Overall, the SIEM system became more intelligent and practical for real-world attack detection.

The integration was configured by creating an AbuseIPDB API key, writing a custom Python integration script, setting the correct file permissions, updating the Wazuh ossec.conf with the integration block, defining custom SSH detection rules, and adding AbuseIPDB result handling rules. The Wazuh agent was configured to monitor a custom log file, and testing was performed by injecting sample SSH log events. Rules 100004 and 100005 were successfully triggered when IPs with non-zero abuse confidence scores were detected, confirming that the integration was working correctly.

A minor limitation of this integration is the API request limit associated with the free AbuseIPDB plan. If a large volume of SSH events occurs in a short time, API rate limiting may delay alert enrichment.

In conclusion, the AbuseIPDB integration significantly strengthened the detection accuracy and response efficiency of Wazuh, making the security monitoring system more reliable and intelligence-driven.